/aiko

/aiko

Minimal native aiko/contract-neuron primitive. `aiko` is the public binary; `contract` is the ledger row/type name and compatibility alias for the same shape.

```
aiko "<goal>" --remote
aiko "<goal>" --local
```

The binary is the public verb. It accepts one goal and one execution mode.

The older `contract` binary remains for compatibility/context activation flows. It must behave as an alias over the same aiko shape, not as a second interface.

`--remote` is bound to Unbrowse and x402-native. It defaults to `https://beta-api.unbrowse.ai`, tags declarations as Corbits/Faremeter Flex requests, and forwards `AIKO_X_PAYMENT` / `X_PAYMENT` as `X-PAYMENT`. Without a payment header the server returns a 402 payment requirements envelope. `--local` uses the same neuron shape against the local scoped ledger.

`aiko` guards privacy before network egress. Local declarations are reduced to patterns and meta-patterns: obvious emails, local filesystem paths, and secret-like tokens are replaced with `[email-pattern]`, `[local-path-pattern]`, and `[secret-pointer-pattern]`. The server may resolve pointer values inside typed context, but the local binary does not leak machine-specific private values.

Every declaration is a `neuron` row. The server-side DAG owns verifier selection, child spawning, evidence integration, threshold firing, and recursive propagation to parent and contract-ref neurons.

`parent_id` is the containment edge: each parent wraps the child as a new environment layer. `contract:<id>` is a cross-edge dependency. The Unbrowse LLM integrates context server-side and spawns the next child contract environments.

Prehooks and posthooks are also contracts, carried by environment: `CONTRACT_PREHOOK_ID` / `CONTRACT_PREHOOK` and `CONTRACT_POSTHOOK_ID` / `CONTRACT_POSTHOOK`. They are breakpoints for API, Telegram, local Mac, VM, Turbobox, and Unbrowse-spawned environments.

PreTool context is a server-side stateless event bus over ledger contracts: `CONTRACT_EVENT_BUS_ID` / `CONTRACT_EVENT_BUS`, `CONTRACT_PRETOOL_EVENT_ID` / `CONTRACT_PRETOOL_EVENT`, and `CONTRACT_EVENT_KIND` / `CONTRACT_PRETOOL_KIND`.

Adapters are server-side contracts too. Local environments carry only `CONTRACT_ADAPTER_ID` / `CONTRACT_ADAPTER` and adapter kind pointers. The Unbrowse LLM compiles the user's prompt text into the root declaration plus the child contract graph, including adapter, hook, and verifier contracts.

For `aiko`, the compiler is recursive and same-shaped: one `aiko "<goal>" --remote` prompt becomes a root aiko contract, and the Unbrowse LLM emits child prompts as more aiko contracts that may themselves carry child prompts. The local binary still sends only one sanitized goal; every child is appended server-side as a `neuron` row with a `parent_id` edge.

Aiko truth is metric-gated. A node resolves true only when the declared goal is true against evaluator aikos bound to external metrics, ledger metrics, or API metrics. When the goal and its evaluators resolve true, the solved aiko is promoted upward as a parent-layer `contract:<id>` pointer. That pointer is the child's typed stdout for the parent aiko that requested it. The compiler is stateless: prompt in, typed tree out; state lives in the append-only ledger.

Posthook webhooks are declared as pointers on the contract. After promotion, Unbrowse POSTs the full contract context tree to `posthook_pointer` and appends a `called` row with `call_kind=posthook`. That is how local and remote aikos call each other after resolution: write the webhook pointer into the contract, then call the pointer after the contract resolves.

The reduced vocabulary is: `aiko` for the primitive/environment, `contract` for the row type, `parent_id` for containment, `contract:<id>` for dependency pointers, `posthook_pointer` for after-resolution calls, `wallet_identity` for signing identity, `replicated_from_contract_id` for paid out-of-lineage copies, and `network_tunnel_contract_id` for WireGuard/Tailscale/WARP-style reachability adapters.

`UNBROWSE_BASE` routes declarations to the Unbrowse server ledger. Local `CONTRACTS_STORE` is only for scoped tests and migrations. Server-spawned environments may carry private verifier wiring in `CONTRACT_NEURON_ACTION`; the public command still stays `aiko "<goal>" --remote|--local`.